

67. Le verre qui se brise

Quatre éclats, et trois variables.

1. Ce qu'il faut comprendre

La chute est celle de la leçon du même nom : les pixels dans `y`, les seizièmes dans `frac`, la pesanteur qui s'ajoute à la vitesse. Ce qui change ici, c'est **ce qui arrive au contact**.

Un objet qui se brise ne disparaît pas : il est **remplacé par ses morceaux**. Au choc, le verre part se cacher et quatre éclats prennent sa place, exactement à l'endroit où il s'est posé.

Cacher un lutin ne demande aucune fonction : on l'écrit à 160, hors des 144 lignes de l'écran. Il existe toujours — les quarante existent toujours — on choisit simplement de ne pas le voir. C'est pour cela que les cinq `sprite()` de la fin sont écrits une fois pour toutes, sans un seul `if` : ce sont les variables qui décident qui est visible.

On ne simule pas quatre éclats. Une image plus tôt ils étaient un seul objet : ils partent donc ensemble, à la même hauteur, et ne diffèrent que par ce qu'ils prennent de côté. Une seule hauteur `ey`, un seul compteur `t`, et les quatre positions se calculent — `76 - 3 * t`, `76 - t`, `76 + t`, `76 + 3 * t`. **Trois variables au lieu de douze**.

Leur envol est la vitesse décalée du saut : `ZEROV - 6` les projette vers le haut, la pesanteur les rattrape. Les deux éclats rapides retombent trois fois plus loin que les lents, sans qu'on ait rien à écrire pour ça — même temps de vol, vitesse triple.

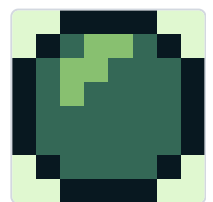
`t` n'avance **que tant qu'ils volent**. Posés, il s'arrête, et les éclats restent exactement où ils sont tombés. C'est cette ligne-là, et aucune autre, qui fait la différence entre du verre brisé et quatre morceaux qui glissent à l'infini.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1769 octets** de cartouche.

1. UN DESSIN — VERRE

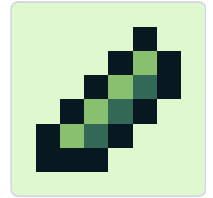
```
Tuile VERRE = {  
  "00333300",  
  "03211230",  
  "32112223",  
  "32122223",  
  "32222223",  
  "32222223",  
  "03222230",  
  "00333300",  
};
```



à l'écran

2. UN DESSIN — ECLAT

```
Tuile ECLAT = {  
    "00000000",  
    "00000300",  
    "00003130",  
    "00031230",  
    "00312300",  
    "03123000",  
    "03330000",  
    "00000000",  
};
```



à l'écran

3. UNE CONSTANCE — CIEL

```
const uint8_t CIEL = 8;
```

4. UNE CONSTANCE — SOL

```
const uint8_t SOL = 128;
```

5. UNE CONSTANCE — CACHE

```
const uint8_t CACHE = 160; // hors de l'écran : un lutin écrit là ne se voit pas
```

6. UNE CONSTANCE — G

```
const uint8_t G = 2;
```

7. UNE CONSTANCE — ZEROV

```
const uint8_t ZEROV = 16; // la vitesse d'un éclat, décalée : 16 est l'immobilité
```

8. UNE CONSTANCE — ATTENTE

```
const uint8_t ATTENTE = 60;
```

9. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    uint8_t y = CIEL;
    uint8_t frac = 0;
    uint8_t vy = 0;

    uint8_t brise = 0;
    uint8_t t = 0;          // images de vol - c'est lui qui écarte les éclats
    uint8_t ey = CACHE;    // leur hauteur, la même pour les quatre
    uint8_t evy = ZEROV;
    uint8_t pose = 0;

    texte(0, 2, "IL TOMBE ET IL CASSE");
    texte(0, 17, "#####");

    while (true) {
        image();

        if (brise) {
            evy = evy + 1;
            if (evy < ZEROV) ey = ey - ZEROV + evy;
            if (evy > ZEROV) ey = ey + evy - ZEROV;

            if (ey >= SOL) {
                ey = SOL;
                evy = ZEROV;
            } else {
                t = t + 1;
            }

            pose = pose + 1;
            if (pose > ATTENTE) {
                brise = 0;
                pose = 0;
                t = 0;
                ey = CACHE;
                y = CIEL;
                frac = 0;
                vy = 0;
            }
        } else {
            vy = vy + G;
            frac = frac + vy;
            y = y + (frac >> 4);
            frac = frac & 15;

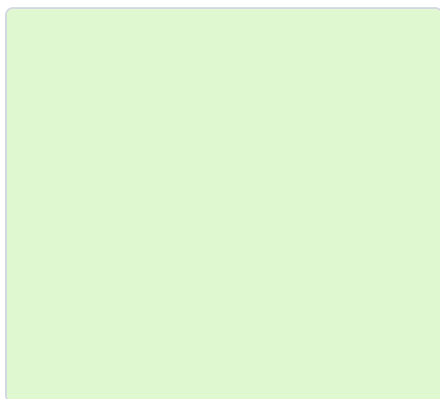
            if (y >= SOL) {
                brise = 1;
                y = CACHE;
                ey = SOL;
                evy = ZEROV - 6;
            }
        }

        sprite(0, 76, y, VERRE);
        sprite(1, 76 - 3 * t, ey, ECLAT);
        sprite(2, 76 - t, ey, ECLAT);
        sprite(3, 76 + t, ey, ECLAT);
        sprite(4, 76 + 3 * t, ey, ECLAT);
    }

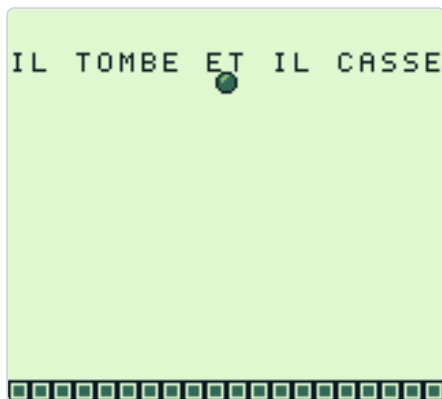
    return 0;
}
```

3. Ce qui se passe, étape par étape

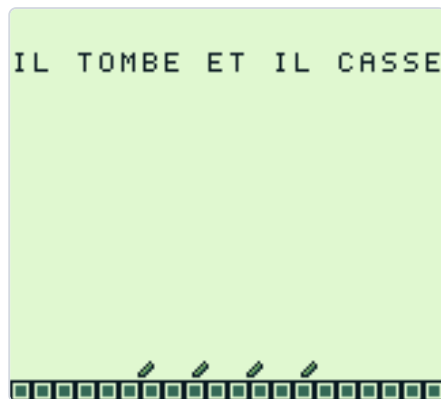
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60

4. Ce qu'on doit voir

Le verre tombe, éclate en quatre morceaux qui s'envolent et retombent éparpillés — puis tout recommence.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le verre tombe, puis s'efface au choc (29 images de chute)
- ✓ quatre éclats prennent sa place
- ✓ ils s'envolent au-dessus du sol ($y = 116$)
- ✓ ils s'écartent des deux côtés (46, 66, 86, 106)
- ✓ les rapides retombent trois fois plus loin (10 et 30 pixels)
- ✓ et tout se pose au sol

Ensuite : **68. Un état de jeu** — niveau 9 sur 10. · gameboy3 — écrire une cartouche Game Boy en C++